

Smart Greenhouse Management System: Phase 3 (Reliability & Integration)

Source Code Repository: [greenhouse-management-dsa-lab](#)

1. Executive Summary

This document outlines Phase 3 of the Smart Greenhouse Management System. In this phase, we established a complete end-to-end distributed architecture by connecting the public-facing REST API gateway to the internal gRPC service, backed by a MongoDB database. Crucially, this phase introduces **Fault Tolerance Mechanisms**, specifically **Retries** and a **Circuit Breaker**, to prevent cascading failures and ensure system resilience when internal components become unresponsive.

2. Deployment Guide

The entire distributed stack (REST API, gRPC backend, and MongoDB) is orchestrated via Docker Compose.

To start the system:

```
docker compose up -d --build
```

Verify all containers are running successfully:

```
docker compose ps
```

3. System Validation & Threat Scenarios

Scenario 1: Normal Operation (Happy Path)

Under normal conditions, the REST API successfully receives HTTP requests, translates them into gRPC calls, and returns the response from the internal backend.

Create a new Greenhouse Item:

```
curl -i -X POST http://localhost:8085/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Sunflower", "status": "healthy", "location": "Zone 1"}'
```

Expected Output (201 Created):

```
HTTP/1.1 201 CREATED
Server: Werkzeug/3.1.8 Python/3.12.13
Date: Mon, 13 Jul 2026 19:33:40 GMT
Content-Type: application/json

{"created_count":1,"message":"created through gRPC","total_count":7}
```

Retrieve the Greenhouse Inventory:

```
curl -i http://localhost:8085/items
```

Expected Output (200 OK):

```
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.12.13
Date: Mon, 13 Jul 2026 19:33:42 GMT
Content-Type: application/json

[
  {"id":"0a066f91-06e3-40c5-b5cc-af25fe54d047","location":"Zone
1","name":"Sunflower","status":"healthy"}
  // ... additional items ...
]
```

Scenario 2: Fault Tolerance Testing (gRPC Backend Down)

To verify our reliability mechanisms, we intentionally simulate a catastrophic failure of the internal gRPC backend to observe how the REST API handles the outage.

1. Induce Failure: Stop the internal gRPC service to simulate a crash or network partition.

```
docker compose stop grpc-service
```

2. First Request - Retry Exhaustion (502 Bad Gateway): Immediately after stopping the service, we send a request. The REST API attempts to reach the backend, retries a configured number of times, and eventually fails gracefully rather than crashing.

```
curl -i -X POST http://localhost:8085/items \
-H "Content-Type: application/json" \
-d '{"name": "Sunflower", "status": "healthy", "location": "Zone 1"}'
```

Expected Output:

```
HTTP/1.1 502 BAD GATEWAY
Content-Type: application/json
```

```
{"error":{"code":"backend_failure","message":"gRPC service did not respond
successfully after retries."}}
```

3. Subsequent Requests - Circuit Breaker Open (503 Service Unavailable): Because the previous attempts failed, the Circuit Breaker "opens." Subsequent requests are instantly rejected without attempting to contact the backend, preventing unnecessary network strain and allowing the backend time to recover.

```
curl -i -X POST http://localhost:8085/items \
-H "Content-Type: application/json" \
-d '{"name": "Sunflower", "status": "healthy", "location": "Zone 1"}'
```

Expected Output:

```
HTTP/1.1 503 SERVICE UNAVAILABLE
Content-Type: application/json
```

```
{"error":{"code":"backend_unavailable","message":"gRPC service is
temporarily unavailable (Circuit Open)."}}}
```

4. System Recovery (Self-Healing): We restart the gRPC service to simulate a system recovery.

```
docker compose start grpc-service
```

After a brief cooldown period (approx. 30 seconds), the Circuit Breaker transitions to a "half-open" state, tests the connection, and upon success, "closes." The system resumes normal operation automatically.

```
HTTP/1.1 201 CREATED
Content-Type: application/json
```

```
{"created_count":1,"message":"created through gRPC","total_count":9}
```

4. Architectural Reflections

- **Preventing Cascading Failures:** Without a circuit breaker, the REST API would continually wait for timeouts from the dead gRPC service. This would quickly consume all available REST worker threads, causing the public-facing API to freeze entirely (a cascading failure).

- **Fail-Fast Paradigm:** By returning a **503 Service Unavailable** immediately when the circuit is open, the system fails fast. This provides immediate feedback to the client (or Automated Watering Controller) that it should try again later, rather than hanging indefinitely.